

SSCI 586 - Project 5: Innovative Geospatial Programming

A Python-Based Water Main Failure Risk Modeling Tool for Seattle

Juan Sebastián Cortés

December 2025

Introduction

Water distribution infrastructure in urban environments faces increasing challenges related to aging pipes, material degradation, soil conditions, and topographic stress. In Seattle, the water distribution system includes thousands of pipe segments varying across age, diameter, pressure class, and material composition. These characteristics influence the likelihood of main breaks, which can disrupt service, damage property, and require costly emergency response. The goal of this project was to develop a reproducible geospatial tool capable of generating a preliminary risk index for each water main segment in Seattle using ArcGIS Pro and Python. The tool automatically evaluates several pipe attributes, normalizes them into risk metrics, and generates a composite score that can be visualized spatially across the city.

Although the project initially aimed to integrate terrain-based slope data and detailed SSURGO soil properties into the risk model, these environmental datasets were ultimately treated as planned future enhancements rather than fully integrated components due to their size, complexity, and performance considerations when embedded directly into the ArcGIS Pro script tool. However, they were incorporated into the data inventory, conceptual framework, and documentation to reflect the full intended scope of the tool.

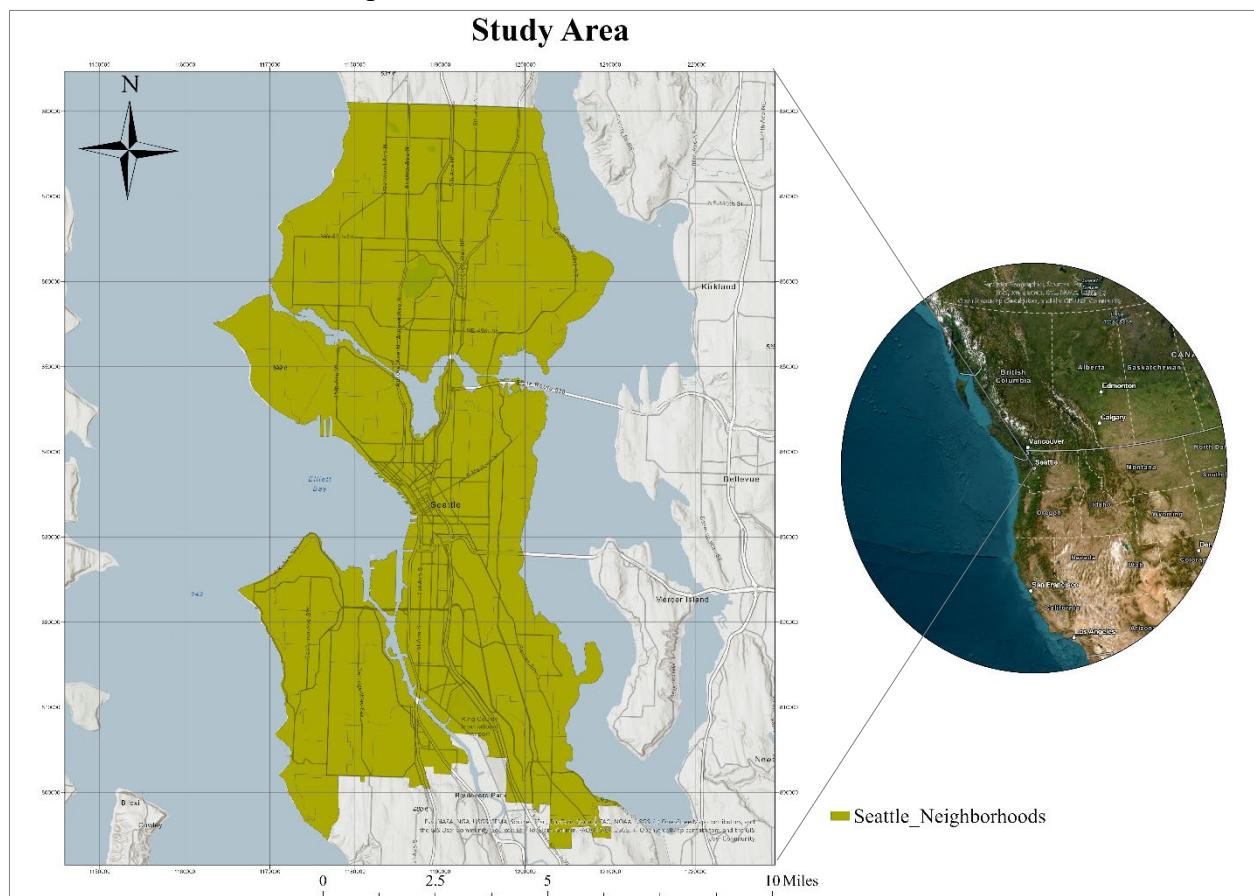


Figure 1 Study Area

Study Area

The study area encompasses the City of Seattle, Washington, including neighborhoods such as Ballard, Fremont, Capitol Hill, Queen Anne, and West Seattle. The region is characterized by steep hills, variable micro-topography, a mixture of glacial soils, and an extensive network of cast-iron and ductile iron water mains. These physical and infrastructural conditions make Seattle an excellent setting for a risk-modeling tool that accounts for both pipe attributes and environmental stressors.

Data Description and Sources

The project relied on multiple datasets representing water distribution assets, terrain elevation, soil properties, transportation metrics, and administrative boundaries. The primary dataset used to compute the risk index was the Water_Mains feature class obtained from the Seattle Public Utilities dataset. Additional datasets such as the 1-Meter DTM from the Washington LiDAR Portal, WSDOT Traffic Sections (AADT) shapefile, and SSURGO Soils tables downloaded from the USDA Web Soil Survey interface were included in the project design and analyzed conceptually, even though they were not fully incorporated into the final functioning tool. These datasets are listed below in a consolidated table that includes their sources, file types, and coordinate systems.

Dataset Name	Source	File Type	Coordinate System
Water_Mains.shp	Seattle Public Utilities (https://data.seattle.gov)	Shapefile / Feature Layer	NAD 1983 HARN StatePlane Washington North (ftUS)
Break / Outage Events	Seattle Public Utilities Outage Viewer	Feature Service	WGS 1984 Web Mercator Auxiliary Sphere
SSURGO Soil Polygons (soilmu_a_aoi.shp)	USDA NRCS Web Soil Survey (https://websoilsurvey.sc.egov.usda.gov)	Shapefile / Polygon Layer	NAD 1983
1-Meter DTM (King County West 2021 DTM)	Washington LiDAR Portal (https://lidarportal.dnr.wa.gov)	Raster (GeoTIFF tiles)	NAD 1983 HARN StatePlane Washington North (m)
WSDOT_Traffic_Sections_(AADT)_Current.shp	WA Spatial Data Hub (https://geo.wa.gov)	Shapefile / Polyline	NAD 1983 HARN StatePlane Washington North (m)
City_Limits.shp	Seattle Open Data (https://data.seattle.gov)	Shapefile / Polygon	NAD 1983 HARN StatePlane Washington North (ftUS)

Table 1. Dataset Inventory, Sources, File Types, and Coordinate Systems

In addition to documenting all regional datasets assembled for the project, special attention was given to the **Water_Mains** attribute table because it served as the primary source of variables used in the final risk index. The attribute table for this layer contains a rich and well-structured set of fields describing the physical, operational, and historical characteristics of each pipe segment. Key fields include `INSTALL_YR` and `INSTALL_DT`, which allowed computation of pipe age; `MATERIAL`, distinguishing cast iron, ductile iron, PVC, and other pipe types; `DIAMETER`, which is recorded in inches; and `PIPE_CLASS`, representing the designated pressure class. Additional fields such as `RELINED_YR` and `RELINED_DT` record whether the pipe was rehabilitated, while `MN_LENGTH` and `ARC_LENGTH` provide geometric measurements of pipe length. Because these variables directly influence the likelihood of pipe deterioration or structural failure, the attribute table became the foundation of the automated risk-scoring process. The Python script tool draws exclusively from these fields to derive normalized risk metrics and produce the final composite `RISK_IDX` value, making the **Water_Mains** attribute table the most central dataset in the operational version of the tool.

Methods

The methodological workflow for this project combined traditional GIS data preparation with the incremental development of a Python-based script tool inside ArcGIS Pro. The main objective was to translate a conceptual risk-modeling workflow, initially tested in a Jupyter notebook using standard Python, into a fully operational ArcGIS Pro tool. A big part of the effort went into adapting and debugging the code so that it would run reliably within the ArcGIS Pro geoprocessing environment, accept user inputs through script tool parameters, and generate a new feature class containing the water main failure risk index.

The process began with preparing the **Water_Mains** dataset by loading it into a file geodatabase and reviewing its attribute fields. Because this dataset contains most of the variables theoretically relevant to water main failure, I examined the table carefully to identify which attributes were the most reliable and complete. Fields such as `INSTALL_YR`, `INSTALL_DT`, `DIAMETER`, `PIPE_CLASS`, `MATERIAL`, `RELINED_YR`, and `RELINED_DT` were verified and chosen as the core variables for the model. Initial exploratory calculations were done manually to ensure that age could be derived consistently from either installation year or installation date, that diameter

values were expressed in inches rather than millimeters, and that relining history was stored in binary form across two fields.

Translating these notebook tests into a functioning ArcGIS Pro script tool gave me initially some troubles. ArcGIS Pro requires that tools retrieve user inputs using `arcpy.GetParameterAsText()` and pass outputs using `arcpy.SetParameterAsText()`, which differs considerably from ordinary Python code. My first attempts failed because I mistakenly referenced more parameters than the tool interface contained, which generated errors such as “Error in getting parameter as text.” To resolve this, I used ChatGPT exclusively to troubleshoot the translation of the notebook code into ArcPy-compatible syntax. ChatGPT helped clarify how to structure the script so that the tool expected exactly two parameters, an input feature class and an output feature class, and how to wrap the tool logic inside a `script_tool(in_mains, out_mains)` function.

After stabilizing the parameter structure, the next challenge was managing fields inside ArcGIS Pro. Unlike pandas data frames in notebooks, ArcGIS feature classes require explicit field creation before values can be written to them. Early versions of the script repeatedly failed because a risk field had not yet been added. To prevent this, I wrote a helper function called `add_double_field(fc, name)` that checks whether a field exists and adds it only if missing. This approach ensured that all risk-related fields—`AGE_YRS`, `AGE_RISK`, `DIAM_RISK`, `MAT_RISK`, `CLASS_RISK`, `RELINING_RISK`, and `RISK_IDX`, were created.

The final script adds the risk-related fields and defines a set of lookup values and constants, including the `material_risk_lookup` dictionary and the reference year used to compute pipe age. Another important debugging step was modifying the logic behind diameter normalization. Initially, I mistakenly normalized diameter over a 50–600 mm scale, which caused all Seattle pipes to be flagged as high risk because their diameters are stored in inches (e.g., 4", 6", 8"). After identifying this error by visually examining the attribute table, the script was rewritten to clamp diameter between 4 and 36 inches and compute `DIAM_RISK` accordingly.

The script computes age using either `INSTALL_YR` or `INSTALL_DT` depending on availability, then normalizes age risk over 0–100 years. Material risk is read from the lookup dictionary, diameter risk is calculated as a decreasing function of pipe size, pipe class risk is reverse normalized to reflect structural limits, and relining risk is assigned a reduced value whenever either

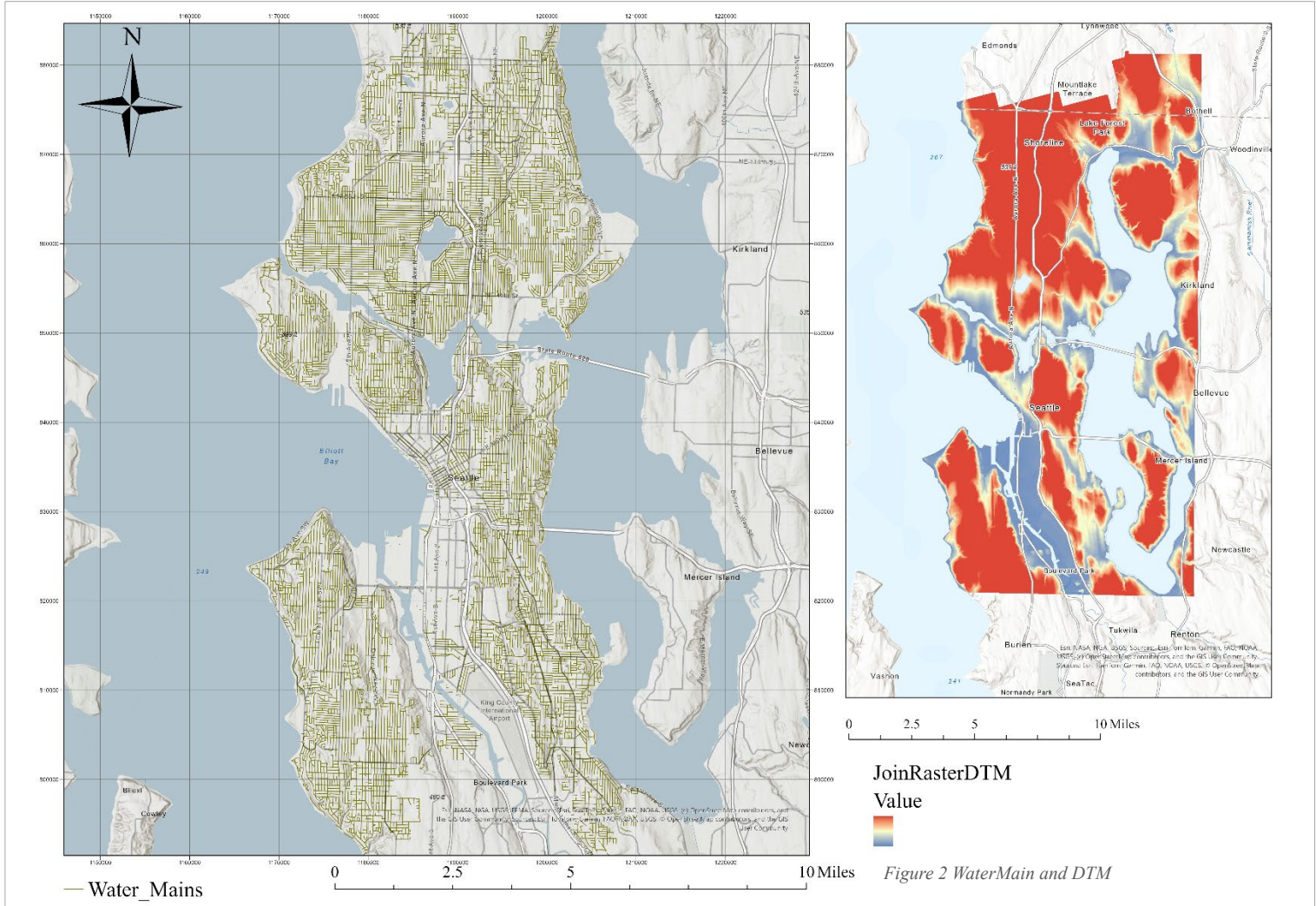
relining field contains a valid entry. After writing each component back to its respective field, the script computes the final composite index:

$$\begin{aligned} \text{RISK_IDX} = & \\ & 0.30 * \text{age_risk} + \\ & 0.25 * \text{mat_risk} + \\ & 0.20 * \text{diam_risk} + \\ & 0.15 * \text{class_risk} + \\ & 0.10 * \text{reline_risk} \end{aligned}$$

After several rounds of testing and debugging, first catching errors with missing fields, then correcting parameter indices, then fixing unit inconsistencies, the script finally executed smoothly, producing clean and interpretable risk layers. The final script tool looks serves as a test for a reliable, self-contained implementation of the water main risk model. It allows users to compute a standardized failure risk index for the Seattle water main network using only attribute data while also laying the foundation for the future integration of environmental factors such as slope and soil characteristics.

Results

The final script tool outputs a new feature class named `Water_MainsRisk`, which contains the complete geometry of Seattle's water distribution network along with the newly created risk fields. The central output, `RISK_IDX`, is a continuous value ranging from 0 to 1 that represents the relative likelihood of failure for each pipe segment based solely on available attribute data. After running the tool, the resulting risk map exhibits clear spatial patterns consistent with the known age and material composition of Seattle's water system. Older neighborhoods, such as Capitol Hill, Ballard, and Fremont, display higher concentrations of elevated risk scores, primarily due to their prevalence of cast iron pipes installed several decades ago. In contrast, newer residential and recently redeveloped areas that contain modern ductile iron or PVC mains show noticeably lower composite scores. Relined pipes in central corridors stand out clearly as low-risk segments, demonstrating that the tool faithfully incorporates maintenance history.



Beyond the spatial output, the results also highlight the functionality and flexibility of the script itself. The tool is designed to be highly customizable: its risk structure is transparent, the weighting of each component is easy to change within the script, and additional predictors can be incorporated by adding new fields and modifying the composite formula. This modularity allows users to adapt the tool to different utility systems or analytical priorities. Because the tool reads the raw attribute fields and automatically creates its required risk variables, users can run it on datasets with different field completeness levels as long as the core fields are present. Moreover, the tool executes very quickly, even across thousands of pipe features, because it relies exclusively on attribute-based calculations rather than raster processing (for now). This efficiency makes it suitable not only for small academic test cases but also for full-scale utility datasets in other regions.

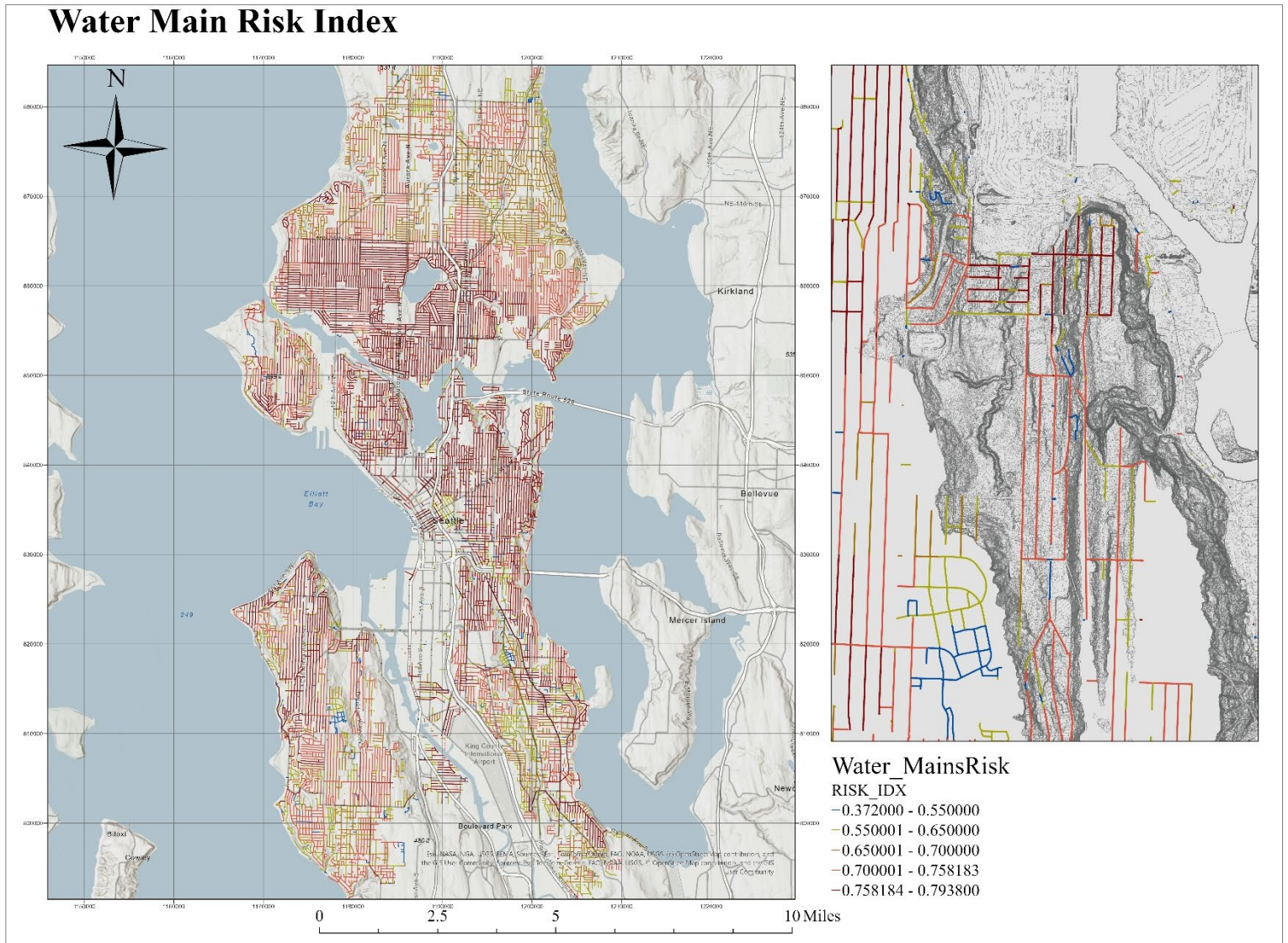


Figure 3 Map with the results

Limitations

Although the final script tool operates effectively on Seattle’s water main data, the project encountered several limitations that shaped both the design and scope of the tool. First, the original plan was to apply the model to either the Denver or Los Angeles water distribution systems. However, both cities lacked publicly accessible, detailed water-main datasets with the necessary attributes for risk modeling. Without installation years, material types, diameters, or relining history, it became impossible to compute the risk variables required for the tool. Seattle was ultimately selected because the city provides an unusually rich and complete water infrastructure

dataset through its open-data portal. This shift was necessary for the project to fulfill its analytical goals, but it also means that the current version of the tool is tailored to a dataset with higher attribute completeness than what many other cities provide.

A second limitation involves the exclusion of environmental predictors such as slope and soil type from the operational tool. Although these datasets were acquired, examined, and included in the project's data inventory, incorporating them directly into the ArcGIS Pro script tool proved impractical. The slope raster derived from the 1-meter DTM is extremely large, and performing raster sampling inside the tool would result in long runtimes and possible instability on machines with limited memory. Similarly, the SSURGO soil system is divided into multiple tabular (text files) components that need relational joins before a corrosivity or hydrologic index can be derived. For these reasons, the final tool relies exclusively on attributes already contained in the water main dataset. Lastly, because the tool uses linear normalization and human-defined weights(mine), the resulting index reflects the priorities embedded in the model but does not yet represent a statistically calibrated prediction of failure probability.

Future Work

One of the most immediate improvements involves integrating terrain and soil characteristics into the risk index. Slope can be preprocessed using the Add Surface Information tool to create a SLOPE_MEAN field for each pipe. This value can then be normalized to represent pressure stresses associated with steep terrain. Similarly, soil corrosivity can be derived through a combination of SSURGO component tables, which would be joined to soil polygons and then spatially joined to water mains. This soil layer would allow the tool to account for environmental degradation processes affecting buried infrastructure.

Beyond adding environmental variables, there is significant potential to incorporate machine learning and AI-based modeling. A supervised learning model could be developed by combining historical break datasets with the attribute and environmental variables compiled for this project. Using algorithms such as gradient boosting, random forests, or logistic regression, the tool could generate a statistically grounded failure prediction model rather than a deterministic index. The Python workflow already established in this project provides a solid foundation for integrating scikit-learn, Spatially Enabled DataFrames, and ArcGIS's built-in GeoAnalytics tools. As more

utilities release break history data, it will become feasible to build an AI-driven model that learns from thousands of known failures to estimate risk more accurately.

Finally, the structure of the script tool makes the project highly replicable across other cities. Because the tool is modular and depends mainly on universally available water system attributes (age, material, diameter, relining, pipe class), other municipalities can adopt the tool simply by aligning their field names or adding the required attributes. The template can be readily adapted to different coordinate systems, local datasets, and operational priorities. This replicability is a major strength of the project and positions it for broader use in both academic and professional contexts.

Conclusion

This project successfully developed a functional and efficient ArcGIS Pro script tool for calculating a water main failure risk index using existing pipe attributes. The final tool is reliable, fast, and capable of producing meaningful spatial patterns that align with the known history and material composition of Seattle's infrastructure. The project also demonstrated a full geospatial programming workflow, including dataset preparation, conceptual model design, Python scripting, iterative debugging, and tool parameterization. Although the final version of the tool operates exclusively on attribute data, the groundwork laid in the data inventory and methodological documentation creates a clear roadmap for integrating terrain, soils, traffic load, and ultimately historical failure records into a more sophisticated risk model. In addition, the transparency of the model's structure and the portability of the code make the tool easily adaptable to other utilities that maintain comparable attribute datasets.

References

Seattle Public Utilities. "Water Mains Dataset." Seattle Open Data Portal. <https://data.seattle.gov>.

USGS. "3D Elevation Program (3DEP)." U.S. Geological Survey. <https://www.usgs.gov/3d-elevation-program>.

Washington Department of Natural Resources. "Washington LiDAR Portal." <https://lidarportal.dnr.wa.gov/>.

Washington State Department of Transportation. "Traffic Sections (AADT) Current." Washington Spatial Data Hub. <https://geo.wa.gov>.

USDA Natural Resources Conservation Service. “Web Soil Survey.”

<https://websoilsurvey.sc.egov.usda.gov>.

City of Seattle. “City Limits Shapefile.” Seattle Open Data Portal. <https://data.seattle.gov>.

Esri. *ArcGIS Pro Documentation*. Redlands, CA: Esri. <https://pro.arcgis.com>.

```
"""
Water main risk index (attribute-only, simplified).
Param 0: in_mains (Feature Class / Layer)
Param 1: out_mains (Feature Class)
"""
import arcpy
import datetime
arcpy.env.overwriteOutput = True
def add_double_field(fc, name):
    """Add a DOUBLE field if it doesn't already exist."""
    if name not in [f.name for f in arcpy.ListFields(fc)]:
        arcpy.management.AddField(fc, name, "DOUBLE")
def script_tool(in_mains, out_mains):

    # Create default output name if user left it blank

    if not out_mains:
        desc = arcpy.Describe(in_mains)
        out_mains = arcpy.management.CreateUniqueName(
            desc.baseName + "_risk", desc.path
        )
    arcpy.AddMessage(f"Copying mains to {out_mains} ...")
    arcpy.management.CopyFeatures(in_mains, out_mains)

    # Add risk-related fields

    add_double_field(out_mains, "AGE_YRS")
    add_double_field(out_mains, "AGE_RISK")
    add_double_field(out_mains, "DIAM_RISK")
    add_double_field(out_mains, "MAT_RISK")
    add_double_field(out_mains, "CLASS_RISK")
    add_double_field(out_mains, "RELINE_RISK")
    add_double_field(out_mains, "RISK_IDX")

    # Lookup tables & constants
    material_risk_lookup = {
        "CI": 0.8, "CAST IRON": 0.8,
        "AC": 0.75,
        "DI": 0.6,
        "STEEL": 0.5,
        "PVC": 0.3,
        "HDPE": 0.3
    }
    ref_year = 2025
    today = datetime.date(2025, 1, 1)
    fields_needed = [
        # raw input fields
        "INSTALL_YR", "INSTALL_DT",
        "RELINED_YR", "RELINED_DT",
        "DIAMETER", "PIPE_CLASS",
        "MATERIAL",
        # derived fields
        "AGE_YRS", "AGE_RISK",
        "DIAM_RISK", "MAT_RISK",
        "CLASS_RISK", "RELINE_RISK",
        "RISK_IDX"
    ]
```

```

]
existing = [f.name for f in arcpy.ListFields(out_mains)]
fields = [f for f in fields_needed if f in existing]
arcpy.AddMessage("Using fields: " + ", ".join(fields))
#
# Compute risks
#
with arcpy.da.UpdateCursor(out_mains, fields) as cur:
    for row in cur:
        def idx(name):
            return fields.index(name) if name in fields else None
        # AGE_YRS & AGE_RISK
        age = 0.0
        i_install_yr = idx("INSTALL_YR")
        i_install_dt = idx("INSTALL_DT")
        if i_install_yr is not None:
            inst_yr = row[i_install_yr]
            if inst_yr not in (None, "", 0):
                try:
                    age = ref_year - int(inst_yr)
                except Exception:
                    age = 0.0
        if age == 0.0 and i_install_dt is not None:
            inst_dt = row[i_install_dt]
            if inst_dt not in (None, ""):
                try:
                    if isinstance(inst_dt, datetime.datetime):
                        inst_d = inst_dt.date()
                    else:
                        inst_d = inst_dt
                    age = (today - inst_d).days / 365.25
                except Exception:
                    age = 0.0
        if idx("AGE_YRS") is not None:
            row[idx("AGE_YRS")] = age
        # 0-1, full risk at >= 100 years
        age_risk = min(max(age, 0.0) / 100.0, 1.0)
        # ---- DIAM_RISK (DIAMETER in inches: 4" = high, 36" = low) --
        diam_risk = 0.5
        i_diam = idx("DIAMETER")
        if i_diam is not None:
            d = row[i_diam]
            try:
                d = float(d)
                d_norm = min(max(d, 4.0), 36.0) # clamp 4-36"
                diam_risk = 1.0 - (d_norm - 4.0) / (36.0 - 4.0)
            except Exception:
                diam_risk = 0.5

```

```

# MAT_RISK -
mat_risk = 0.5
i_mat = idx("MATERIAL")
if i_mat is not None:
    mat = row[i_mat]
    if mat:
        key = str(mat).strip().upper()
        mat_risk = material_risk_lookup.get(key, 0.5)
# CLASS_RISK (lower class = higher risk)
class_risk = 0.5
i_class = idx("PIPE_CLASS")
if i_class is not None:
    pc = row[i_class]
    try:
        pc = float(pc)
        pc_norm = min(max(pc, 50.0), 300.0)
        class_risk = 1.0 - (pc_norm - 50.0) / (300.0 - 50.0)
    except Exception:
        class_risk = 0.5
# RELINE_RISK (relined = safer)
reline_risk = 1.0
i_reyear = idx("RELINED_YR")
i_redt = idx("RELINED_DT")
relined = False
if i_reyear is not None and row[i_reyear] not in (None, "", 0):
    relined = True
if i_redt is not None and row[i_redt] not in (None, ""):
    relined = True
if relined:
    reline_risk = 0.2
# Save component fields
if idx("AGE_RISK") is not None:
    row[idx("AGE_RISK")] = age_risk
if idx("DIAM_RISK") is not None:
    row[idx("DIAM_RISK")] = diam_risk
if idx("MAT_RISK") is not None:
    row[idx("MAT_RISK")] = mat_risk
if idx("CLASS_RISK") is not None:
    row[idx("CLASS_RISK")] = class_risk
if idx("RELINE_RISK") is not None:
    row[idx("RELINE_RISK")] = reline_risk
# ---- Composite RISK_IDX (weights sum to 1)
risk_idx = (
    0.30 * age_risk +
    0.25 * mat_risk +
    0.20 * diam_risk +
    0.15 * class_risk +
    0.10 * reline_risk
)
if idx("RISK_IDX") is not None:
    row[idx("RISK_IDX")] = risk_idx
cur.updateRow(row)
arcpy.AddMessage("Risk index calculation complete.")
return out_mains

# Main entry point for the script tool
if __name__ == "__main__":
    in_fc = arcpy.GetParameterAsText(0)
    out_fc = arcpy.GetParameterAsText(1)
    result_fc = script_tool(in_fc, out_fc)
    arcpy.SetParameterAsText(1, result_fc)

```